HDOC Day-69 Activity

Generating Fibonacci Numbers Recursively and Counting Recursive Calls

Student Name	Dhruv Pandey
SAP ID	590037660
Batch	15
Course / Activity	HDOC Day-69

Aim

To write a C program that generates the first **n Fibonacci numbers** using a recursive function and also counts the total number of calls made to that recursive Fibonacci function.

Fibonacci Sequence

The Fibonacci sequence starts with **0, 1**. Every later term is the sum of the previous two terms. Thus the sequence begins: 0, 1, 1, 2, 3, 5, 8, 13, ...

Problem Requirements

- Read the number of Fibonacci terms n.
- Generate the first n Fibonacci numbers using recursion.
- Use the recursive relation F(n) = F(n-1) + F(n-2).
- Count every invocation of the recursive Fibonacci function.
- Display the Fibonacci sequence and the total recursive-call count.

Recursive Idea

For index 0, Fibonacci is 0. For index 1, Fibonacci is 1. For every index greater than 1, the function recursively calculates the two previous Fibonacci values and adds them. A counter is increased immediately whenever the function is entered.

Algorithm

1. Read n.
2. Initialize a global variable **call_count = 0**.
3. For each Fibonacci index from 0 to n-1, call **fibonacci(i)** and display the returned value.
4. Every time fibonacci() begins, increment call_count.
5. If n == 0, return 0. If n == 1, return 1.
6. Otherwise return **fibonacci(n-1) + fibonacci(n-2)**.
7. After all n terms are generated, display call_count.

C Program

```c
#include <stdio.h>

long long call_count = 0;

long long fibonacci(int n)
{
    call_count++;

    if (n == 0)
        return 0;

    if (n == 1)
        return 1;

    return fibonacci(n - 1) + fibonacci(n - 2);
}

int main()
{
    int n;

    printf("Enter number of terms: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Please enter a positive integer.");
        return 0;
    }

    printf("Fibonacci series: ");

    for (int i = 0; i < n; i++)
    {
        printf("%lld", fibonacci(i));

        if (i < n - 1)
            printf(" ");
    }

    printf("\nTotal recursive calls: %lld", call_count);

    return 0;
}
```

How the Call Counter Works

The statement **call_count++** is the first statement inside fibonacci(). Therefore, every function invocation is counted, including base-case calls such as fibonacci(0) and fibonacci(1). The counter accumulates calls made while generating all terms from F(0) through F(n-1).

Dry Run - fibonacci(4)

Call	Expansion / Return
fib(4)	fib(3) + fib(2)
fib(3)	fib(2) + fib(1)
fib(2)	fib(1) + fib(0)
fib(1)	returns 1

fib(0)	returns 0

The recursion tree contains repeated calculations. For example, fib(2) is evaluated more than once while computing fib(4). Each such invocation contributes separately to the recursive-call count.

Sample for n = 5

Index	Fibonacci value	Calls for this fib(i)
0	0	1
1	1	1
2	1	3
3	2	5
4	3	9
Total	0 1 1 2 3	19 calls

For n = 5, the program calls fibonacci(0), fibonacci(1), ..., fibonacci(4) separately to print the first five terms. With the naive recursive definition shown above, the accumulated number of calls is **19**.

Complexity

Time Complexity: Exponential for naive recursive Fibonacci; computing a high-index term takes approximately $O(2^n)$ time in the usual upper-bound analysis because many subproblems are recalculated.

Space Complexity: O(n) for the maximum recursion depth.

Result

The program successfully generates the first n Fibonacci numbers using recursion and counts every invocation of the recursive Fibonacci function. The call count also demonstrates the repeated work performed by the basic recursive method.

Execution Example

```
Enter number of terms: 5
Fibonacci series: 0 1 1 2 3
Total recursive calls: 19
```